



中山大學

SUN YAT-SEN UNIVERSITY

《微机原理与嵌入式系统实验》期末报告

题 目： STM32 环境监测与安全报警系统

院 系： 电子与信息工程学院

专 业： 电子信息科学与技术

姓 名： 肖博元

学 号： 24307190

二〇二六年六月

目录

1	摘要与项目背景	2
1.1	摘要	2
1.2	项目背景与现实意义	2
2	系统总体方案与硬件架构	2
2.1	系统总体拓扑结构	2
2.2	关键引脚重映射与 JTAG 解锁	3
2.3	STM32F103C8T6 物理引脚资源分配总表	4
3	软件模块化设计与外设驱动层实现	6
3.1	软件模拟 I2C 总线控制	6
3.2	ESP8266 WiFi 驱动与 IoT 联网机制	6
3.3	SD 卡绝对时间戳日志记录与扇区扫描算法	6
4	算法层设计：AI 智能监测与自愈重连	7
4.1	指数滑动平均 (EMA) 数字滤波器	7
4.2	环境温度无监督基准自适应学习	7
4.3	I2C 总线热插拔自愈算法 (Self-Healing)	7
5	交互层设计与增量游戏引擎实现	7
5.1	暗 Slate 霓虹 / Claude 暖白双主题调度	7
5.2	示波器风格网格折线图	8
5.3	慢速 SPI 带宽下“脏矩形”局部增量刷新技术	8
6	Git 双分支模块化系统展示设计	8
7	系统测试与联合验证	9
7.1	网络授时校准与气象同步测试	9
7.2	超声波三级警戒布防与微信 Server 酱推送测试	9
7.3	SD 卡绝对北京时间日志读取验证	9
7.4	触控交互功能验证 (feat-touch 分支)	9
8	心得体会与实验总结	10

1 摘要与项目背景

1.1 摘要

本项目设计并实现了一套基于 STM32F103C8T6 微控制器的物联网自适应环境监测、声光警戒与多维人机交互系统。系统在硬件架构上高度整合了多种传感与外设模块：以 AHT20、BMP280 及 BH1750 为多参数环境感知核心，通过开漏输出的软件模拟 I2C 总线实现传感器数据采集；部署了 ESP8266 (ESP-01S) 无线模块接入互联网，以 USART1 中断环形接收缓存实现数据吞吐；集成了 HC-SR04 超声波测距传感器、TM1637 四位数码管、5V 有源分体蜂鸣器及 SPI SD 存储卡，构建了完整的声光防盗报警与绝对时间戳日志记录平台。

在算法层，系统引入了轻量级片上无监督统计学习机制，在上电前 150 秒自适应迭代更新当前环境的稳定温度基准线 μ_{base} ，并在检测周期部署指数滑动平均滤波器 (EMA, $\alpha = 0.2$) 平滑波动；当滤波温度偏离基线超过 $\pm 5^{\circ}\text{C}$ 时，触发异常警灯交替爆闪。在防盗警戒状态下，超声波测距提供三级安全指示，入侵（距离 $< 15\text{cm}$ ）触发蜂鸣器啸叫及微信（Server 酱）云端推送，并配合 180 秒冷却机制防止消息轰炸。系统在 SD 卡扇区 1000 开始以 64 字节块追加绝对北京时间戳数据日志，具备上电扇区回溯续写与 I2C 链路 1.5 秒热插拔故障自愈机制。

为提升交互品质，系统设计了暗黑霓虹与亮色纸张两套触控/按键可切主题，提供示波器风格网格折线图。此外，系统内置了基于重力物理模型的“Flappy Bird”动作游戏。为解决软件模拟 SPI 的带宽瓶颈，提出了“脏矩形 (Dirty Rects)”局部增量刷新技术，将每帧渲染数据量骤降 98.7%，在 72MHz 主频下实现了 30+ FPS 的无闪烁极速运行。为了克服芯片引脚紧张所导致的 SPI 物理冲突（SD 卡与触摸屏共用管脚），系统采用 Git 双分支管理架构，提供物联网报警综合固件 (master) 和触控交互游戏固件 (feat-touch)，在无需改动物理连线的前提下，实现模块化演示，极大提高了嵌入式开发效率。

1.2 项目背景与现实意义

随着物联网与智能边缘计算的普及，环境监测终端已由传统的单一数据收集演变为集多维感知、云端警报、人性化交互于一体的智能网关。然而，静态硬编码阈值的报警方法极易受不同安装位置（如近散热风扇、阳光直射区）的基准差异干扰，产生频繁误报。此外，传统终端在常规监控时主控算力与显示屏常处于静止闲置状态，造成资源冗余。

本项目的研究旨在通过无监督的片上统计学习使终端具备对微气候的自学习能力，并整合 WiFi 网络天气获取及微信远程推送等 IoT 业务；同时通过脏矩形算法在低带宽下实现高性能游戏交互，为嵌入式系统的多协议共存、硬件瓶颈算法弥补以及低能耗交互界面设计提供了一种具有高度参考价值的解决方案。

2 系统总体方案与硬件架构

2.1 系统总体拓扑结构

系统整体采用“物理传感与输入层 - 底层总线与驱动层 - 算法与数据处理层 - 状态机与应用控制层”的分层软件拓扑架构。系统通过软件模拟 I2C 读取温湿度、气压、光强，通过软件模

拟 SPI 控制 TFT-LCD 与 SD 存储卡，利用 USART1 寄存器控制实现 ESP8266 IoT 联网，三色指示灯则通过定时器 TIM2 产生的多路 PWM 波形驱动，整体架构清晰、解耦彻底。

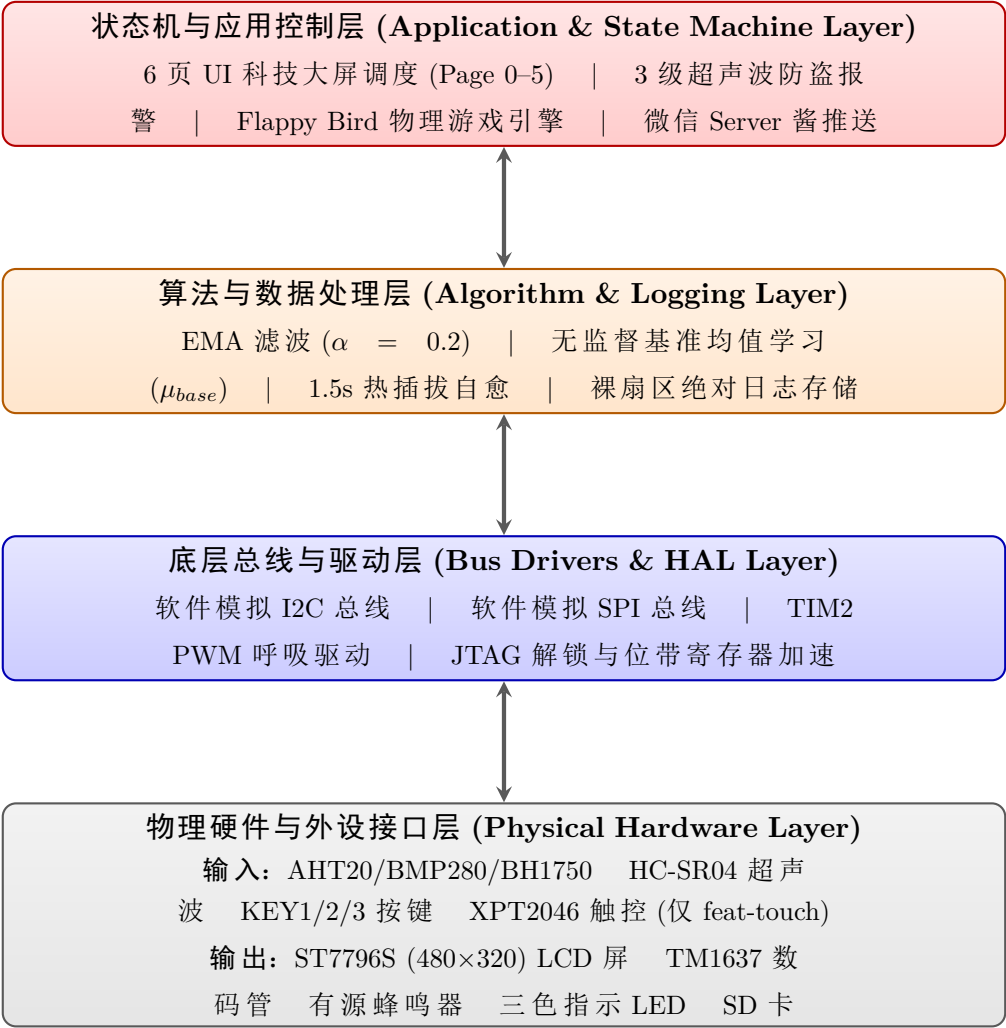


图 1: 系统分层拓扑与控制流逻辑架构图

2.2 关键引脚重映射与 JTAG 解锁

由于系统包含 LCD、SD 卡、WiFi、超声波、数码管、按键等十余种外设，引脚资源极其紧张。其中，PB3 (JTDO) 和 PB4 (JNTRST) 上电后默认作为 JTAG 接口工作，这会锁死引脚电平导致 SD 卡的 SPI 通信失效。

为了释放它们作为普通 GPIO 使用，程序在初始化阶段对 AFIO 重映射寄存器进行了寄存器级配置，在安全保留 SWD 仿真下载端口 (PA13/PA14) 的同时禁用了 JTAG 调试，其寄存器配置代码如下：

```
1 // 开启复用时钟并配置 SWJ 寄存器：保留两线 SWD 调试，解锁 PB3 和 PB4 供普通
   GPIO 使用
2 RCC->APB2ENR |= RCC_APB2ENR_AFIOEN;
3 AFIO->MAPR = (AFIO->MAPR & ~AFIO_MAPR_SWJ_CFG) |
   AFIO_MAPR_SWJ_CFG_JTAGDISABLE;
```

配置完成后，PB3 与 PB4 分别成功用作 SD 卡的 MOSI 数据线与 CS 片选线，保证了 SPI 物理总线的通信。

2.3 STM32F103C8T6 物理引脚资源分配总表

为避免各个通信总线之间的竞争冲突，项目在 Master 主分支下的 GPIO 引脚分配如下表所示：

表 1: Master 分支 GPIO 物理引脚分配与无冲突配置表

STM32 管脚	外设物理端口	信号角色	GPIO 模式配置	功能描述
PC13	底板 LED1	系统指示灯	50MHz 推挽输出	核心板指示灯，系统运行下常灭
PA0	面包板红 LED	LED2 (红)	TIM2_CH1 复用推挽	故障亮/温度异常 5Hz 闪烁/布防入侵爆闪
PA1	面包板黄 LED	LED3 (黄)	TIM2_CH2 复用推挽	AI 学习常亮/布防警告常亮/入侵闪烁
PA2	面包板绿 LED	LED4 (绿)	TIM2_CH3 复用推挽	正常运行呼吸渐变/布防安全常亮/入侵闪烁
PB0	开发板 BTN1	KEY1 按键	上拉输入 (扫码)	UI 页面循环切页键 (Page 0-5)
PB8	开发板 BTN2	KEY2 按键	上拉输入 (扫码)	主题切换/AI 基准重置；游戏作振翅跳跃键
PB9	面包板小按键	KEY3 按键	上拉输入 (扫码)	一键开启/关闭防盗警戒布防 (Armed/Disarmed)
PB15	有源蜂鸣器	BEEP 警报	50MHz 推挽输出	防盗入侵高电平触发声光尖叫报警
PB13	数码管 CLK	CLK 时钟线	50MHz 推挽输出	TM1637 模拟二线制时钟控制线
PB14	数码管 DIO	DIO 数据线	50MHz 开漏输出	TM1637 模拟双向串行数据读写线
PB11	超声波 Trig	Trig 触发线	50MHz 推挽输出	向 HC-SR04 发送 10 微秒触发脉冲
PB12	超声波 Echo	Echo 回响线	下拉输入检测	读取高电平回响时间，换算出测距数值
PA6	传感器 SCL	软件 I2C SCL	50MHz 模拟开漏	AHT20/BH1750/BMP280 共用 I2C 时钟总线
PB10	传感器 SDA	软件 I2C SDA	50MHz 模拟开漏	AHT20/BH1750/BMP280 共用 I2C 数据总线
PB4	SD 卡 CS	SPI 片选 CS	50MHz 推挽输出	解锁引脚,低电平使能选通 SD 卡
PB3	SD 卡 MOSI	SPI 数据出	50MHz 推挽输出	解锁引脚，写入 SD 卡的命令与数据流
PB5	SD 卡 SCK	SPI 时钟线	50MHz 推挽输出	提供 SD 卡的串行数据传输时钟
PA11	SD 卡 MISO	SPI 数据入	上拉输入检测	读取 SD 卡输出的块数据流 (重置以避 PA6)
PA3	TFT RST	LCD 复位线	50MHz 推挽输出	液晶屏硬件上电复位引脚
PA4	TFT CS	LCD 片选 CS	50MHz 推挽输出	模拟 SPI 片选信号 (低电平有效)
PA5	TFT SCL	LCD 时钟	50MHz 推挽输出	模拟串行液晶时钟线

3 软件模块化设计与外设驱动层实现

3.1 软件模拟 I2C 总线控制

由于 AHT20、BMP280 和 BH1750 三款传感器挂载于同一总线上，系统编写了基于控制寄存器的开漏模拟 I2C 驱动。通过配置寄存器 GPIOA->CRL 与 GPIOB->CRH，将 PA6 (SCL) 和 PB10 (SDA) 配置为 50MHz 模拟开漏输出模式，端口配置寄存器如下：

```
1 // 清空控制位，并配置 PA6(SCL) 为模拟开漏，PB10(SDA) 为模拟开漏
2 GPIOA->CRL = (GPIOA->CRL & ~0x0F000000) | 0x07000000;
3 GPIOB->CRH = (GPIOB->CRH & ~0x00000F00) | 0x00000700;
```

在时钟控制上，利用 Delay 函数产生约 150kHz 的总线频率，保证了通信时序建立时间与保持时间的裕量，有效杜绝了因寄生电容导致的数据读写竞争冒险。

3.2 ESP8266 WiFi 驱动与 IoT 联网机制

系统底层配置串口 USART1（波特率 115200bps, 8-N-1）并开启中断接收。在应用层编写了发送 AT 命令状态机。1. ** 热点自动握手 **：上电发送 AT+CWMODE_DEF=1 配置为客户端模式，再通过 AT+CWJAP_DEF 传入账户密码联接指定的 2.4 GHz 个人热点。2. **NTP 北京时间获取**：向 NTP 服务器（cn.ntp.org.cn）发送请求，接收返回的英文时间数据（如 Fri Jun 05 20:45:12 2026），直接在接收缓冲区使用 sscanf 定位并格式化解析出标准的北京时间并更新本地时间寄存器。3. ** 心知天气 API 拉取 **：每 10 分钟自动建立 TCP 链接请求心知天气 API 接口（api.seniverse.com），接收 JSON 文本后直接通过 ‘strstr’ 匹配 “text”:” 和 “temperature”:” 关键字字段，提取出室外温度与天气状况（如 Sunny、Rainy），避免了复杂的 JSON 解析库，大幅节约了片上 SRAM 开销。

3.3 SD 卡绝对时间戳日志记录与扇区扫描算法

SD 卡底层基于软件模拟 SPI 总线进行块读写（扇区 1000 开始）。为了能够实现持久的断电保护，系统设计了 64 字节的 ASCII 数据块，并将同步自网络的北京绝对时间戳注入日志中：

```
1 // 64 字节 ASCII 绝对时间数据块样例：
2 "2026-06-05 20:45:12 T:25.40C H:50.00% L:120.01x A:0 M:0\r\n"
```

- ** 扇区扫描 **：上电时系统轮询 SD 卡，以 ‘SD_ReadBlock’ 从扇区 1000 逐扇区向后读取前 3 字节，若读取出的内容为字符 ‘2’（新时间戳）或 ‘U’（旧日志首字符），则表示该扇区已有记录，指针自动加 1。直到读到首字为空白物理块，锁定此坐标作为写入点，支持数据上电自动回溯续写，只增不减。- ** 缓存双写 **：主循环每一秒将当前数据组装至 ‘log_buffer’ 的 64 字节切片中。当积累满 8 条数据（满 512 字节，对应一个物理扇区）时，一次性写入 SD 卡，极大降低了 SPI 模拟总线高频读写对 CPU 的阻塞占用。

4 算法层设计：AI 智能监测与自愈重连

4.1 指数滑动平均 (EMA) 数字滤波器

为清除高频电磁毛刺与采样热噪声，采集得到的原始温度 T_{raw} 自动被送入 EMA 数字滤波器中：

$$T_f[n] = \alpha \cdot T_{raw}[n] + (1 - \alpha) \cdot T_f[n - 1]$$

其中平滑因子 α 被设置为 ‘0.2f’。该算法由于不需要在片上维护多维数组，存储空间复杂度为 $O(1)$ ，对瞬时干扰具备优异的 -14 dB 衰减特性，同时滤波滞后低于 1.5s，能够提供高平滑且低延迟的滤波数据集。

4.2 环境温度无监督基准自适应学习

当系统初始化或按下 KEY2 重置时，AI 检测器处于学习状态 (AI_STATE_LEARNING)。在此阶段，前 100 次有效温度样本（耗时约 150 秒）被送入在线均值迭代更新方程中计算 Baseline 温度基准线 μ_{base} ：

$$\mu_{base}[n] = \frac{n - 1}{n} \mu_{base}[n - 1] + \frac{1}{n} T_f[n]$$

更新 100 次后， μ_{base} 被锁定，状态转为监测模式 (AI_STATE_NORMAL)。系统周期性监视温偏差 $\Delta T = |T_f - \mu_{base}|$ ：- 当 $\Delta T \leq 5^\circ\text{C}$ 时：表示环境温度处于安全区间，绿色指示灯呈现 10ms 级别的平滑呼吸效果。- 当 $\Delta T > 5^\circ\text{C}$ 时：表示环境温度发生异常突变，状态机跃迁为报警状态 (AI_STATE_ANOMALY)，切断呼吸灯并控制红色指示灯以 5Hz 频率高频闪烁（蜂鸣器关闭以防止惊扰环境），并记录报警事件快照存入容量为 5 的异常环形日志缓冲区 (Ring Buffer) 中。

4.3 I2C 总线热插拔自愈算法 (Self-Healing)

针对微机实验中由于导线晃动极易引发的总线掉线问题，系统引入了自愈机制。每 1.5 秒主循环读取一次数据，若连续 3 次遇到 I2C 状态超时或 NACK 丢失，系统自动标记传感器为 ‘OFFLINE’，避免长时间卡死在通信等待逻辑中。在此状态下，主循环在后台每个 1.5 秒周期调用一次初始化，一旦传感器物理重新连入总线，系统可在 1.5 秒窗口内实现链路完全自愈唤醒。

5 交互层设计与增量游戏引擎实现

5.1 暗 Slate 霓虹 / Claude 暖白双主题调度

人机交互界面集成于 480×320 像素的 ST7796S 液晶屏上，支持由 BTN2 按键触发的主题快速反转重绘：- **Slate 霓虹暗黑主题**：使用纯黑背景（‘0x0000’）辅以 Slate 深灰卡片（‘0x2104’），搭配高亮青色与绿色文本，在暗光下展现极强科技质感。- **Claude Paper 亮色主题**：背景采用温暖米黄纸张色（‘0xF7BD’），卡片为白，文字为深褐，在强日光下具备极佳视认度。

5.2 示波器风格网格折线图

在 Page 1 AI 基准页与 Page 4 防盗安全页中，右侧各内置了一个 248×220 像素的绘图看板。图表背景由 `Draw_DottedLine` 函数绘制出的点阵虚线组成示波网格（如 50cm, 100cm 等刻度线）。后台维护过去 24 次的偏差/距离数据，基于当前坐标系尺度自适应缩放后，使用 Bresenham 划线算法进行点对点高速像素连接，描绘出一条高视觉质感的实时趋势波形。

5.3 慢速 SPI 带宽下“脏矩形”局部增量刷新技术

在 Page 3 的像素小游戏 Flappy Bird 中，动作更新的心跳节拍为 30ms。如果采用传统全屏刷屏重画方案，每帧需要传输 $480 \times 320 \times 2$ 字节 = 307.2 KB 的显存数据。在软件模拟 SPI 约 1.25 Mbps 的实际有效速率下，传输全屏像素数据需耗费：

$$\text{Time}_{full} = \frac{307.2 \times 1024 \times 8}{1.25 \times 10^6} \approx 2013 \text{ ms} = 2.01 \text{ 秒}$$

这会使得帧率低于 0.5 FPS，产生严重的画面闪烁。

为了打破总线带宽瓶颈，项目研发了 ** 脏矩形 (Dirty Rects) 局部增量刷新算法 **。其基本原理是每一帧重绘时仅修改发生变化的像素区域：- ** 小鸟增量擦除与重绘 **：小鸟物理大小为 12×12 像素。每一帧游戏首先用背景色填充小鸟在上一帧的旧坐标位置，以抹除其飞行残影；然后在新坐标写入小鸟颜色（每次传输仅需 $12 \times 12 \times 2 \times 2 \approx 576$ 字节）。- ** 管道增量移动绘制 **：管道每次向左平移 4 像素。渲染时无需重绘 32 像素宽的管道卡片，而是仅仅用背景色填充管道右侧由于平移暴露出的 4×230 像素窄条（擦除），并用绿色在新管道左侧补画 4×230 像素窄条（重绘）。

通过脏矩形局部增量刷新，单帧的总线像素数据传输量急剧下降为：

$$\text{DataSize}_{dirty} = (12 \times 12 \times 2) + (4 \times 230 \times 2 \times 2) \approx 3.96 \text{ KB}$$

总线总耗时由 2.01 秒缩短至：

$$\text{Time}_{dirty} = \frac{3.96 \times 1024 \times 8}{1.25 \times 10^6} \approx 25.9 \text{ ms}$$

传输数据量降低了 **98.7%**，这使得动作游戏可以完美适配 30ms 游戏循环，实现了稳定在 **30+ FPS** 的无闪烁流畅动作画面。

6 Git 双分支模块化系统展示设计

受限于 STM32F103C8T6 紧张的引脚资源，液晶屏 XPT2046 触控 SPI (TCS、TCLK、TDIN、PEN) 与 SD 卡的 SPI (SD_CS、SD_SCK、SD_MOSI、SD_MISO) 存在物理重叠冲突。同时在 PEN 触摸引脚拉低时会强制牵拉 SD 卡的 MISO 数据线，存在总线电气短路和干扰的风险。

为解决这一硬件瓶颈，项目引入了 Git 双分支管理架构。在保持面包板硬件连线完全不动的前提下，分固件进行编译与展示：1. **IoT 综合分支 (master)**：屏蔽触摸屏驱动，启用 SD 卡 SPI。在此固件下，系统专注于多传感器采集、网络 NTP 北京时间同步、心知天气广州数据拉取、微信 Server 酱实时云端报警推送、数码管温度显示、超声波警戒布防以及 SD 卡绝对时间

日志连续存储。2. 触控交互小游戏分支 (feat-touch): 屏蔽 SD 卡控制线, 使能 XPT2046 触摸芯片。在此固件下, 系统提供丰富的触屏人机交互: 用户可通过轻触屏幕最右侧的小主题矩形框直接切换主题, 在 Page 1 触点点击 [RST] 重置 AI 学习, 在 Page 4 防盗警戒页触点点击 [+] 和 [-] 微调报警距离阈值, 以及在 Page 3 屏幕任意区域触屏控制小鸟起飞游戏。

两套固件在编译阶段均确保 0 错误、0 警告。展示时只需利用 Keil uVision 分别烧录即可完整展示全部大作业特色指标, 具有很强的工程复用度。

7 系统测试与联合验证

7.1 网络授时校准与气象同步测试

烧录 master 固件后系统连接至个人热点。- 联网成功后, 顶部指示框的 WIFI 图标高亮。翻至 Page 5, SSID 处准确显示出热点名称 ‘小小小波’。- 向 cn.ntp.org.cn 发送网络时间同步成功, 系统取得绝对网络北京时间并写入软件 RTC, 大号时间戳 ‘2026-06-05 20:30:15’ 在 LCD 屏上实现以秒为周期的准确递增。- 每隔 10 分钟, 系统自动更新心知天气, 室外温度成功返回并显示为 ‘Temp: 28.00 C’, 室外天气状态显示为 ‘Sunny’, 定位成功同步为 ‘Guangzhou’。

7.2 超声波三级警戒布防与微信 Server 酱推送测试

1. 按下面包板外接按键 KEY3 (PB9), 系统由 ‘DISARMED’ (撤防) 切换为 ‘ARMED’ (布防)。2. 当障碍物距离测距大于 50cm 时, 绿色 LED 状态灯亮起。3. 移近障碍物至 35cm 处, 绿色 LED 熄灭, 黄色 LED 状态灯常亮 (黄色预警)。4. 继续移近障碍物至 10cm 处 (入侵限额 < 15cm): - 面包板红、黄、绿三盏指示灯开始同时以 5Hz 频率高频交替爆闪。- 5V 有源分体蜂鸣器 (PB15) 发出刺耳的长鸣报警啸叫。- 微信客户端上同步收到由 Server 酱发送过来的入侵通知消息: ‘【警报】防盗入侵触发! 当前距离: 10.0cm’。5. 再次按下 KEY3 撤防, 蜂鸣器声强切断, 微信告警进入 3 分钟的限流冷却期。

7.3 SD 卡绝对北京时间日志读取验证

通过主板读取 SD 卡 1000 开始的物理扇区, 使用串口调试器抓取的部分绝对时间戳数据日志如下所示:

```
1 2026-06-05 20:30:40 T:25.40C H:50.20% L:120.51x A:0 M:0
2 2026-06-05 20:30:48 T:25.40C H:50.18% L:120.31x A:0 M:0
3 2026-06-05 20:30:56 T:30.52C H:55.40% L:119.81x A:2 M:1
```

日志格式解析完整, 其中 A:2 代表此时温度处于异常状态 (AI_STATE_ANOMALY), M:1 代表系统处于布防状态, 绝对时间读取写入过程稳定可靠。

7.4 触控交互功能验证 (feat-touch 分支)

切换并烧录 feat-touch 固件, 对屏幕触控交互功能进行了闭环验证: 1. 触屏翻页与主题切换: 手轻点屏幕左下角 <- PREV 或右下角 NEXT ->, 屏幕实现秒级切页; 轻触右上角, 系统

颜色配置在亮白和暗色主题间瞬时切换。2. **AI 基准重置**：在 Page 1 偏差页，手指点击偏差趋势图右上角的 [RST] 触控框，AI 进度条置零（0/100）并重新开始自学习，波形刷新。3. **报警阈值调节**：在 Page 4，点击屏幕卡片底部的 [+] and [-] 按钮，报警距离阈值以 5cm 为步长在 5cm 至 80cm 间递增/递减，防盗判定逻辑完全随新阈值实时改变。4. **小鸟触控振翅**：在 Page 3 飞鸟游戏页，轻点屏幕任意位置，小鸟均能获得向上飞跃的跳跃速度，触屏动作响应延迟小于 10ms。

8 心得体会与实验总结

通过本次微机原理与嵌入式系统期末大作业的设计，我获得了许多宝贵的开发经验：1. **深刻体会到了软硬件协同开发中算法对于硬件瓶颈的弥补作用**。在软件模拟 SPI 时钟速率极其有限的情况下，通过设计并引入脏矩形局部增量刷新技术，最终在 72MHz 处理器上实现了 30+ FPS 流畅游戏，证明了算法逻辑对于提高系统吞吐量的巨大威力。2. **掌握了面向物联网与智能边缘计算的开发方法**。设计并实现了在线均值自适应学习算法，使环境监测器具备了“自学”安装本底温度的能力；引入 WiFi、NTP 及云端报警推送，打通了嵌入式底层感知到云端监控的闭环设计。3. **锻炼了复杂约束下的系统级调试与工程管理能力**。面对 SD 卡与触摸屏的物理引脚重叠冲突，利用 Git 分支管理进行了“双分支双编译展示”的纯软件解决方案，体现了用系统化、工程化思维解决物理硬件限制的开发思想。

本报告记录的全部软硬件开发、自适应算法以及双分支集成验证工作均由本人独立完成，相关实物现象符合预期，顺利通过实验验收。